Relatório Técnico – Padrões de Projeto GoF

Projeto: Nexus E-Commerce | **Fase:** 2 – Implementação do Núcleo e Padrões GoF (Aula

---

1. Visão Geral da Arquitetura e Objetivo

Este relatório técnico apresenta a justificativa e a implementação de 3 Padrões de Projeto GoF (Gang of Four) no núcleo de negócios do sistema Nexus. Os padrões foram aplicados diretamente nas regras de domínio para resolver problemas de acoplamento, duplicação e rigidez de código:

- **Strategy (Comportamental):** Cálculo flexível de modalidades de frete.
- **Factory (Criacional):** Criação padronizada de cupons de troca e promocionais.
- **Builder (Criacional):** Construção progressiva e validação do objeto Pedido.

2. Padrão 1: Strategy (Comportamental)

2.1. Problema e Motivação

A classe `FreteService` continha o cálculo de frete em um único método com tarifas fixas por UF. A inclusão de novas opções de envio (como frete expresso prioritário ou frete grátis acima de determinado valor) exigiria a criação de múltiplas estruturas condicionais (if-else), violando o princípio Aberto/Fechado (Open/Closed Principle - OCP).

2.2. Solução Implementada

Foi criada a interface `CalculoFreteStrategy`, definindo o contrato comum para os algoritmos de cálculo. Cada modalidade foi isolada em uma classe própria:

- **Interface:** `br.com.java.nexus.patterns.strategy.frete.CalculoFreteStrategy`
- **Implementações Concretas:**
  - `FretePadraoStrategy`: Tarifa regional por UF somada ao peso dos produtos.
  - `FreteExpressoStrategy`: Modalidade expressa com adicional de 60% e prazo de 1 a 2 dias.
  - `FreteGratisPromocionalStrategy`: Frete com valor zero para carrinhos com subtotal a partir de R$ 350,00.
- **Contexto:** A classe `FreteService` recebe as estratégias via injeção de dependência e delega a execução conforme o tipo de frete solicitado.

2.3. Benefícios

Permite adicionar novas modalidades de entrega apenas criando novas classes de estratégia, sem necessidade de modificar a lógica existente de `FreteService`.

3. Padrão 2: Factory (Criacional)

3.1. Problema e Motivação

A instanciação de cupons ocorria de forma dispersa nas regras de pagamento (troco por cupons de troca excedentes), devolução de itens e cupons promocionais. Cada trecho configurava manualmente cerca de 10 atributos (código, prazo de validade de 6 meses, flags de ativação e vínculo com cliente), gerando repetição de código e risco de inconsistência.

3.2. Solução Implementada

Foi criada a classe `CupomFactory` (pacote `br.com.java.nexus.patterns.factory.cupom`) para centralizar a criação dos cupons:

- `criarCupomTrocaExcedente(Cliente, BigDecimal)`: Cria cupom de troca com prefixo "TROCA-", validade de 6 meses e uso único para troco.
- `criarCupomTrocaDevolucao(Cliente, Long devolucaoId, BigDecimal)`: Cria cupom de troca atrelado à devolução aprovada.
- `criarCupomPromocional(String, BigDecimal, ...)`: Cria cupons promocionais para campanhas de marketing com limites de uso.

A classe `CupomService` foi atualizada para utilizar exclusivamente a `CupomFactory` para gerar novos cupons.

3.3. Benefícios

Centralização das regras de expiração e códigos de cupom em um único ponto, reduzindo redundâncias e atendendo ao princípio de responsabilidade única (SRP).

4. Padrão 3: Builder (Criacional)

4.1. Problema e Motivação

A entidade `Pedido` é um objeto complexo que exige cliente, endereço de entrega, pagamento autorizado, múltiplos itens do carrinho, frete, cupons e recálculo total. A criação com construtores convencionais tornava o código extenso e sujeito a erros de preenchimento ou pedidos sem itens.

4.2. Solução Implementada

Foi implementada a classe `PedidoBuilder` (pacote `br.com.java.nexus.patterns.builder.pedido`) com métodos encadeados para montagem segura:

```
Pedido pedido = PedidoBuilder.novoPedido()
    .paraCliente(cliente)
    .comEnderecoEntrega(enderecoEntrega)
    .comPagamento(pagamento)
    .comStatus(StatusPedido.EM_PROCESSAMENTO)
    .comFrete(valorFrete)
    .comDescontoPromocional(descontoPromocional)
    .comCupomPromocional(codigoCupomPromocional)
```

```
    .comItensDoCarrinho(carrinho)
    .build();
```

O método `build()` valida as regras mínimas (cliente preenchido, endereço presente, pagamento informado e itens válidos) antes de retornar a instância para o `PedidoService`.

4.3. Benefícios

Código mais claro e legível, eliminando a criação de pedidos em estado incompleto ou inválido.

5. Validação e Testes Automatizados

A validação dos três padrões foi executada por meio de testes unitários na classe `PadroesProjetoGoFTest`:

Padrão GoF	Cenário Testado	Resultado Obtido	Status
Strategy	Cálculo de frete Padrão, Expresso (+60%) e Promocional Grátis (>= R$ 350)	Valores e regras aplicados corretamente por modalidade	Aprovado
Factory	Criação de cupons de troca (excedente/devolução) e promocionais	Formatação correta de prefixo TROCA-, validade de 6 meses e vínculo	Aprovado
Builder	Construção de pedido completo a partir do carrinho	Pedido montado com todos os itens, frete, descontos e total consistente	Aprovado

6. Conclusão

A implementação dos padrões Strategy, Factory e Builder atendeu às regras de negócio do núcleo do sistema Nexus, estruturando o código de acordo com as boas práticas da disciplina de Engenharia de Software / Padrões de Projeto.